

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: #DATA LOADED
data=pd.read_csv("C:\\Users\\Lichh\\Documents\\spam_email1.csv")
df=pd.DataFrame(data)
print(df)
print(df.isnull().sum())
```

```

                                     text label
0    Can we reschedule our Saturday call? I have a ...    ham
1    The quarterly budget review is scheduled for S...    ham
2    Your package is held at customs. Pay $52573 fe...  spam
3    Send this to 5 people and receive $31482 from ...  spam
4    Your PayPal account is limited. Verify immedia...  spam
...
5245  Free trial of our premium service. No credit c...  spam
5246  Quick question ? did you get a chance to look ...    ham
5247  The invoice #445190 for $26630 is attached. Pl...    ham
5248  Double your investment in 24 hours with our se...  spam
5249  Prescription drugs at 80% off. No prescription...  spam

[5250 rows x 2 columns]
text      257
label     261
dtype: int64
```

```
In [3]: print(df.head())
```

```

                                     text label
0    Can we reschedule our Saturday call? I have a ...    ham
1    The quarterly budget review is scheduled for S...    ham
2    Your package is held at customs. Pay $52573 fe...  spam
3    Send this to 5 people and receive $31482 from ...  spam
4    Your PayPal account is limited. Verify immedia...  spam
```

```
In [4]: print("shape:-",df.shape)
```

```
shape:- (5250, 2)
```

```
In [5]: print("\nLabel counts:")
print(df['label'].value_counts())
```

```
Label counts:
label
ham      2499
spam     2490
Name: count, dtype: int64
```

```
In [6]: df = df.dropna()
df = df.drop_duplicates()
df.to_csv("cleaned_spamemail.csv", index=False)
df
```

Out[6]:

	text	label
0	Can we reschedule our Saturday call? I have a ...	ham
1	The quarterly budget review is scheduled for S...	ham
2	Your package is held at customs. Pay \$52573 fe...	spam
3	Send this to 5 people and receive \$31482 from ...	spam
4	Your PayPal account is limited. Verify immedia...	spam
...	...	...
4992	Make \$17345 per day working from home! No expe...	spam
4994	Lose 13 lbs in 18 days GUARANTEED! Our miracle...	spam
4995	Earn \$68838 daily with crypto! Bitcoin investm...	spam
4997	Your package is held at customs. Pay \$26736 fe...	spam
4999	LAST CHANCE: Confirm your subscription or lose...	spam

2654 rows × 2 columns

In [7]: `print(df)`

```
              text label
0  Can we reschedule our Saturday call? I have a ...   ham
1  The quarterly budget review is scheduled for S...   ham
2  Your package is held at customs. Pay $52573 fe...  spam
3  Send this to 5 people and receive $31482 from ...  spam
4  Your PayPal account is limited. Verify immedia...  spam
...
4992  Make $17345 per day working from home! No expe...  spam
4994  Lose 13 lbs in 18 days GUARANTEED! Our miracle...  spam
4995  Earn $68838 daily with crypto! Bitcoin investm...  spam
4997  Your package is held at customs. Pay $26736 fe...  spam
4999  LAST CHANCE: Confirm your subscription or lose...  spam
```

[2654 rows x 2 columns]

In [8]: `print(df.shape)`

(2654, 2)

In [9]: `print(df.isnull().sum())`

```
text      0
label     0
dtype: int64
```

In [10]: `df['label'] = df['label'].str.strip()`

```
In [11]: #converted text labels to numeric values
import pandas as pd
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['label'] = le.fit_transform(df['label'])
```

```
print(dict(zip(le.classes_, le.transform(le.classes_)))
df.to_csv("cleaned_spamemail.csv", index=False)
```

```
{'ham': np.int64(0), 'spam': np.int64(1)}
```

```
In [12]: # Convert text column to lowercase
df['text'] = df['text'].str.lower()
df.to_csv("cleaned_spamemail.csv", index=False)
print("Converted to lowercase successfully!")
```

Converted to lowercase successfully!

```
In [13]: # Removed numbers from text column
df['text'] = df['text'].str.replace(r'\d+', '', regex=True)
df.to_csv("cleaned_spamemail.csv", index=False)
print("Numbers removed successfully!")
```

Numbers removed successfully!

```
In [14]: #Removed punctuations from text column
df['text'] = df['text'].str.replace(r'^\w\s', '', regex=True)
df.to_csv("cleaned_spamemail.csv", index=False)
print("Punctuation removed successfully!")
```

Punctuation removed successfully!

```
In [15]: import pandas as pd
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt_tab')
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
def remove_stopwords(text):
    words = word_tokenize(text)
    filtered = [w for w in words if w.lower() not in stop_words]
    return " ".join(filtered)
df['text'] = df['text'].apply(remove_stopwords)
df.to_csv("cleaned_spamemail.csv", index=False)
print("Stopwords removed successfully!")
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\Lichh\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Lichh\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
Stopwords removed successfully!
```

```
In [16]: print(df.head())
```

	text	label
0	reschedule saturday call conflict morning	0
1	quarterly budget review scheduled sunday pleas...	0
2	package held customs pay fee httpcustomscom	1
3	send people receive bill gates	1
4	paypal account limited verify immediately http...	1

```
In [17]: def remove_short_words(text):
words = text.split()
words = [w for w in words if len(w) > 2]
return " ".join(words)
```

```
df['text'] = df['text'].apply(remove_short_words)
df.to_csv("cleaned_spamemail.csv", index=False)
print("Short words removed successfully!")
```

Short words removed successfully!

```
In [18]: df['text'] = df['text'].str.replace(r'\s+', ' ', regex=True).str.strip()
df.to_csv("cleaned_spamemail.csv", index=False)
print("Extra whitespaces removed successfully!")
```

Extra whitespaces removed successfully!

```
In [19]: X = df['text']
y = df['label']
X
```

```
Out[19]: 0      reschedule saturday call conflict morning
1      quarterly budget review scheduled sunday pleas...
2      package held customs pay fee httpcustomscom
3      send people receive bill gates
4      paypal account limited verify immediately http...
...
4992   make per day working home experience needed join
4994   lose lbs days guaranteed miracle pill works in...
4995   earn daily crypto bitcoin investment guarantee...
4997   package held customs pay fee httpcustomscom
4999   last chance confirm subscription lose access c...
Name: text, Length: 2654, dtype: object
```

```
In [20]: from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
cv = CountVectorizer()
X_train_transformed = cv.fit_transform(X_train)
X_test_transformed = cv.transform(X_test)
```

```
In [21]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_rep
model = LogisticRegression()
model.fit(X_train_transformed, y_train)
y_pred = model.predict(X_test_transformed)
```

```
In [22]: print("Accuracy:", accuracy_score(y_test, y_pred))
print("confusion_matrix:", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.998745294855709
confusion_matrix: [[212  0]
 [ 1 584]]
```

```
In [23]: from sklearn.svm import SVC

# -----
# 1. SVM - Linear Kernel
# -----
svm_linear = SVC(kernel='linear')
svm_linear.fit(X_train_transformed, y_train)

y_pred_linear = svm_linear.predict(X_test_transformed)
print("\nSVM Linear Accuracy:", accuracy_score(y_test, y_pred_linear))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_linear))
```

```

# -----
# 2. SVM - Polynomial Kernel
# -----
svm_poly = SVC(kernel='poly', degree=3)
svm_poly.fit(X_train_transformed, y_train)

y_pred_poly = svm_poly.predict(X_test_transformed)
print("\nSVM Polynomial Accuracy:", accuracy_score(y_test, y_pred_poly))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_poly))

# -----
# 3. SVM - RBF Kernel
# -----
svm_rbf = SVC(kernel='rbf')
svm_rbf.fit(X_train_transformed, y_train)

y_pred_rbf = svm_rbf.predict(X_test_transformed)
print("\nSVM RBF Accuracy:", accuracy_score(y_test, y_pred_rbf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rbf))

```

SVM Linear Accuracy: 0.998745294855709

Confusion Matrix:

```
[[212  0]
 [ 1 584]]
```

SVM Polynomial Accuracy: 0.9974905897114178

Confusion Matrix:

```
[[212  0]
 [ 2 583]]
```

SVM RBF Accuracy: 0.998745294855709

Confusion Matrix:

```
[[212  0]
 [ 1 584]]
```

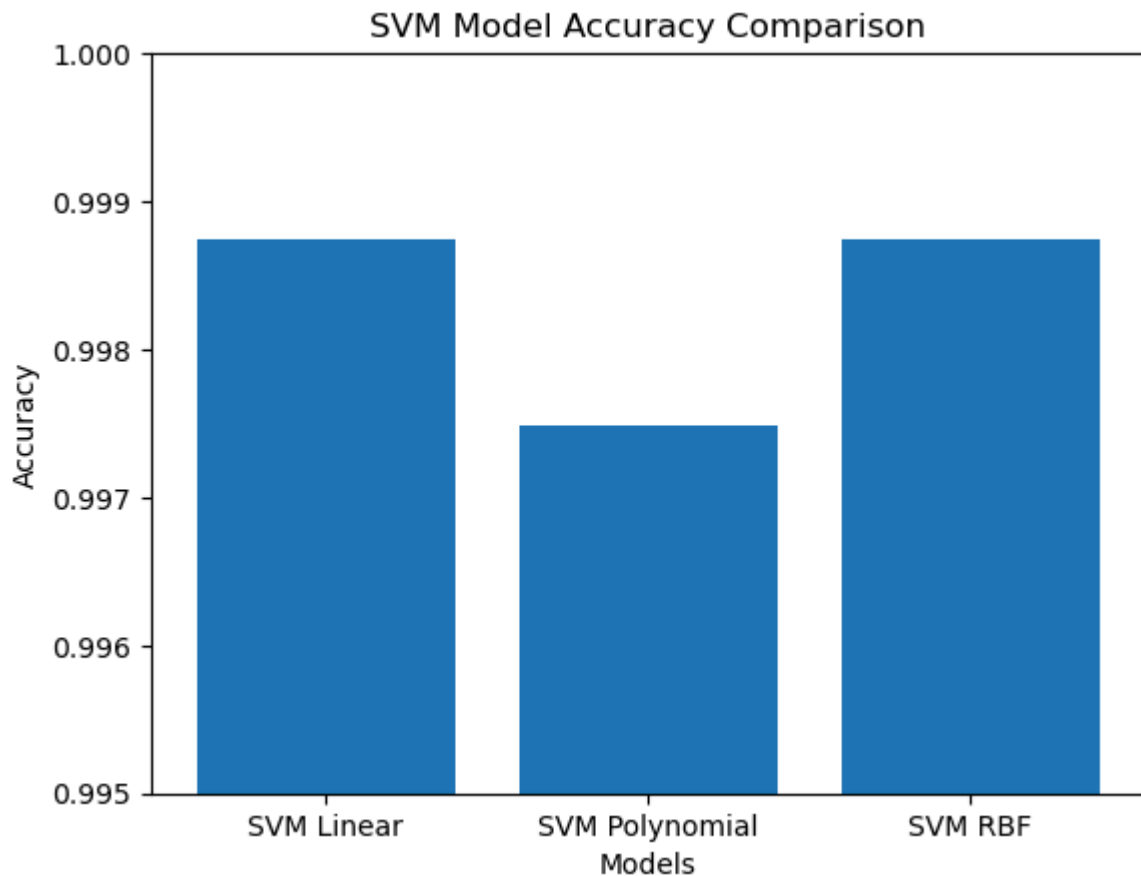
In [24]: `import matplotlib.pyplot as plt`

```

# Accuracies
models = ['SVM Linear', 'SVM Polynomial', 'SVM RBF']
accuracies = [0.998745294855709, 0.9974905897114178, 0.998745294855709]

plt.figure()
plt.bar(models, accuracies)
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("SVM Model Accuracy Comparison")
plt.ylim(0.995, 1.0) # zoom for better view
plt.show()

```



```
In [25]: import pandas as pd

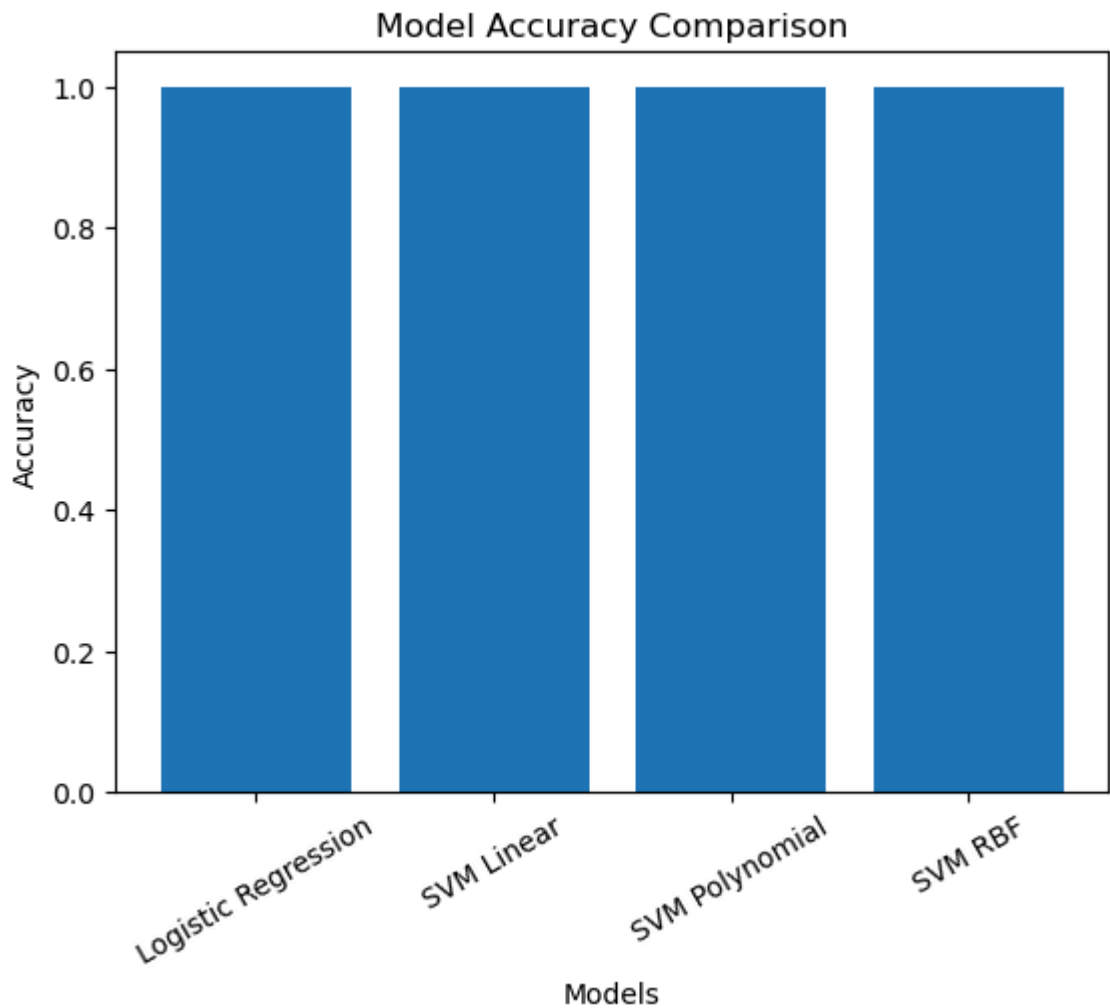
results = {
    "Model": ["Logistic Regression", "SVM Linear", "SVM Polynomial", "SVM RBF"],
    "Accuracy": [
        accuracy_score(y_test, y_pred),
        accuracy_score(y_test, y_pred_linear),
        accuracy_score(y_test, y_pred_poly),
        accuracy_score(y_test, y_pred_rbf)
    ]
}

results_df = pd.DataFrame(results)
print(results_df)
```

	Model	Accuracy
0	Logistic Regression	0.998745
1	SVM Linear	0.998745
2	SVM Polynomial	0.997491
3	SVM RBF	0.998745

```
In [26]: import matplotlib.pyplot as plt

plt.figure()
plt.bar(results_df["Model"], results_df["Accuracy"])
plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.xticks(rotation=30)
plt.show()
```



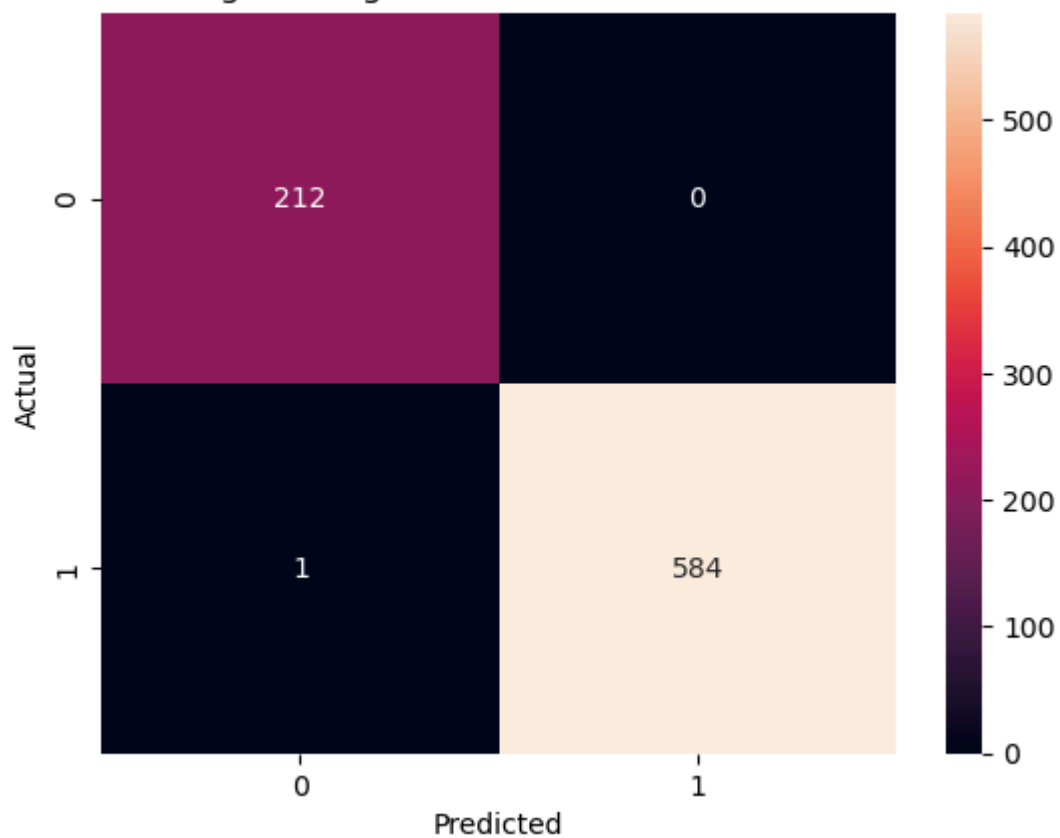
```
In [27]: import seaborn as sns

models = {
    "Logistic Regression": y_pred,
    "SVM Linear": y_pred_linear,
    "SVM Polynomial": y_pred_poly,
    "SVM RBF": y_pred_rbf
}

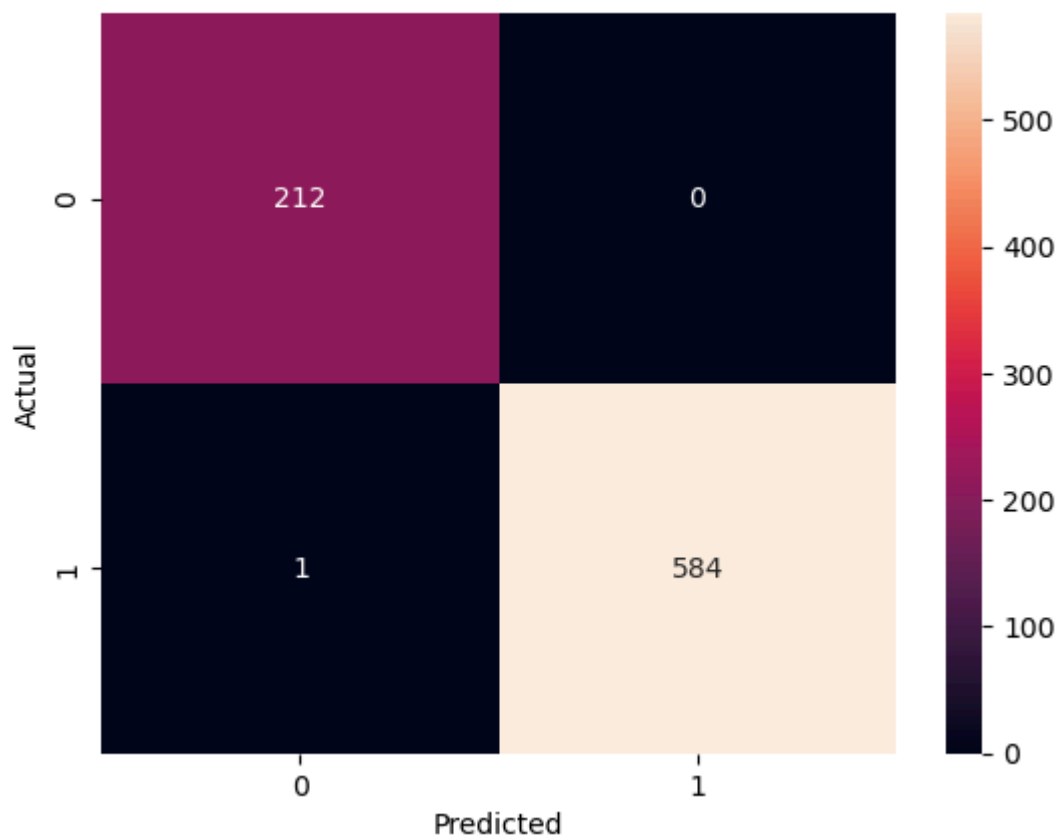
for name, pred in models.items():
    cm = confusion_matrix(y_test, pred)

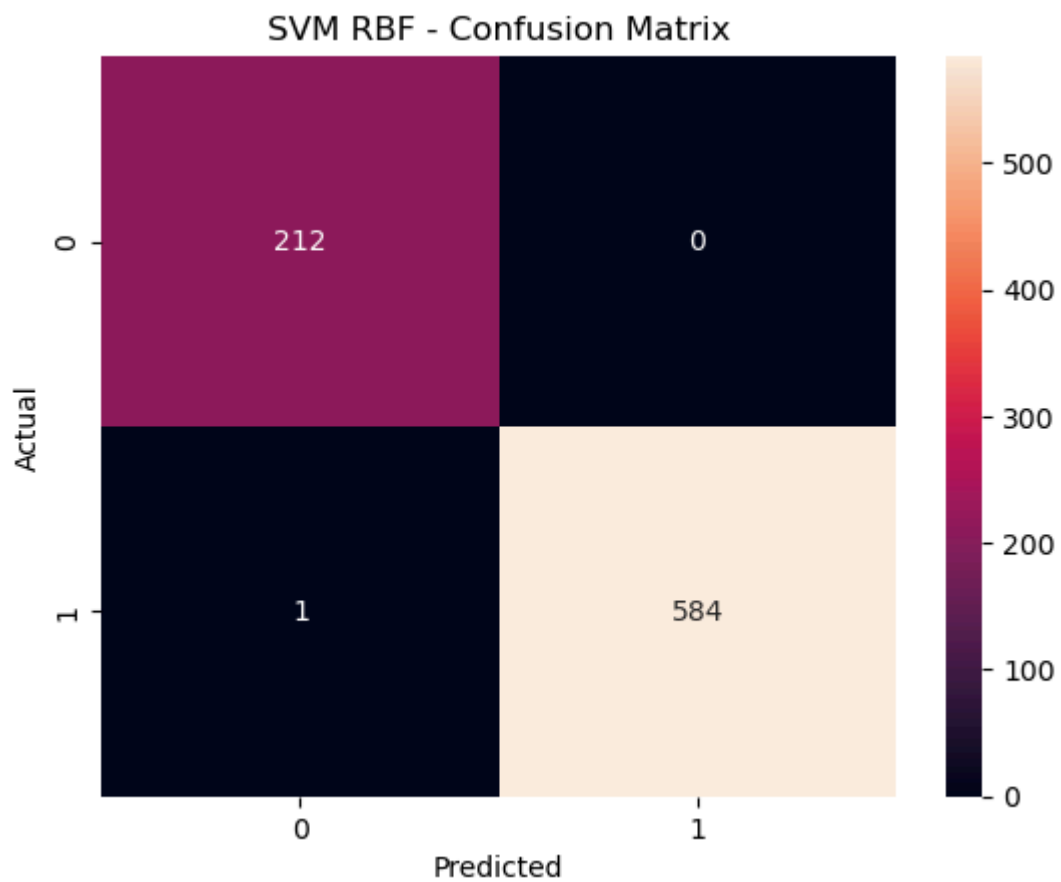
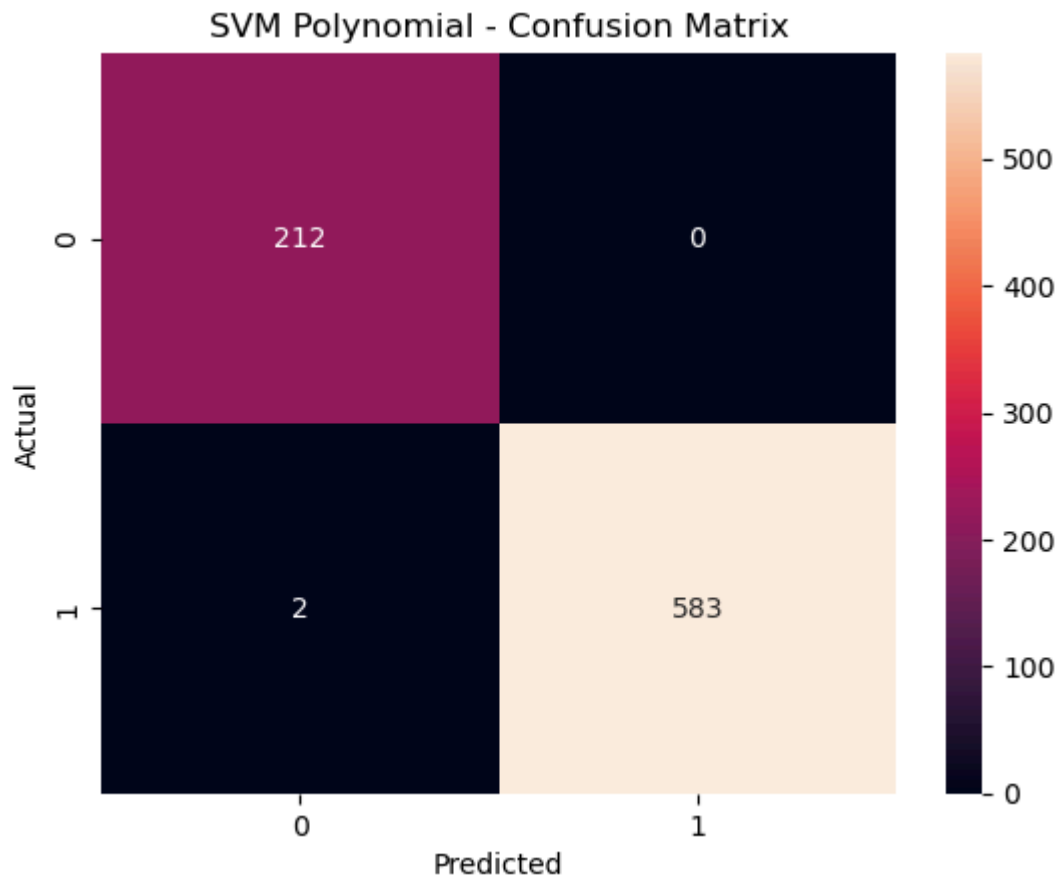
    plt.figure()
    sns.heatmap(cm, annot=True, fmt='d')
    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
```

Logistic Regression - Confusion Matrix



SVM Linear - Confusion Matrix





```
In [28]: for name, pred in models.items():  
          print(f"\n{name} Classification Report:")  
          print(classification_report(y_test, pred))
```

```

Logistic Regression Classification Report:
              precision    recall  f1-score   support

         0              1.00      1.00      1.00        212
         1              1.00      1.00      1.00        585

 accuracy              1.00              1.00              1.00        797
 macro avg              1.00      1.00      1.00              797
 weighted avg           1.00      1.00      1.00              797

```

```

SVM Linear Classification Report:
              precision    recall  f1-score   support

         0              1.00      1.00      1.00        212
         1              1.00      1.00      1.00        585

 accuracy              1.00              1.00              1.00        797
 macro avg              1.00      1.00      1.00              797
 weighted avg           1.00      1.00      1.00              797

```

```

SVM Polynomial Classification Report:
              precision    recall  f1-score   support

         0              0.99      1.00      1.00        212
         1              1.00      1.00      1.00        585

 accuracy              1.00              1.00              1.00        797
 macro avg              1.00      1.00      1.00              797
 weighted avg           1.00      1.00      1.00              797

```

```

SVM RBF Classification Report:
              precision    recall  f1-score   support

         0              1.00      1.00      1.00        212
         1              1.00      1.00      1.00        585

 accuracy              1.00              1.00              1.00        797
 macro avg              1.00      1.00      1.00              797
 weighted avg           1.00      1.00      1.00              797

```

```

In [29]: # Find best model based on highest accuracy
best_model_index = results_df["Accuracy"].idxmax()
best_model_name = results_df.loc[best_model_index, "Model"]
best_accuracy = results_df.loc[best_model_index, "Accuracy"]

print("\nBest Model:", best_model_name)
print("Best Accuracy:", best_accuracy)

# Get confusion matrix of best model
model_preds = {
    "Logistic Regression": y_pred,
    "SVM Linear": y_pred_linear,
    "SVM Polynomial": y_pred_poly,
    "SVM RBF": y_pred_rbf
}

```

```

best_cm = confusion_matrix(y_test, model_preds[best_model_name])

TN, FP, FN, TP = best_cm.ravel()

print("\nConfusion Matrix of Best Model:")
print(best_cm)

print("\nFinal Conclusion:")
print(f"The best-performing model is {best_model_name} with an accuracy of {best")
print(f"It has {FP} False Positives and {FN} False Negatives, indicating strong")
print(f"Therefore, this model is selected as the final model for the given task.")

```

Best Model: Logistic Regression
 Best Accuracy: 0.998745294855709

Confusion Matrix of Best Model:
 [[212 0]
 [1 584]]

Final Conclusion:
 The best-performing model is Logistic Regression with an accuracy of 0.9987.
 It has 0 False Positives and 1 False Negatives, indicating strong classification
 performance.
 Therefore, this model is selected as the final model for the given task.

```

In [30]: import pickle

# -----
# Collect all models and their accuracies
# -----
models = {
    "Logistic Regression": (model, accuracy_score(y_test, y_pred)),
    "SVM Linear":          (svm_linear, accuracy_score(y_test, y_pred_linear)),
    "SVM Polynomial":      (svm_poly, accuracy_score(y_test, y_pred_poly)),
    "SVM RBF":             (svm_rbf, accuracy_score(y_test, y_pred_rbf)),
}

# -----
# Find the best model by highest accuracy
# -----
best_model_name = max(models, key=lambda name: models[name][1])
best_model_obj, best_accuracy = models[best_model_name]

print(f"Best Model : {best_model_name}")
print(f"Best Accuracy: {best_accuracy:.4f}")

# -----
# Save the best model using pickle
# -----
with open("best_model.pkl", "wb") as f:
    pickle.dump(best_model_obj, f)

print("Best model saved as 'best_model.pkl'")

# -----
# (Optional) Save the vectorizer too!
# -----
with open("count_vectorizer.pkl", "wb") as f:
    pickle.dump(cv, f)

```

```
print("CountVectorizer saved as 'count_vectorizer.pkl'")
```

Best Model : Logistic Regression
Best Accuracy: 0.9987
Best model saved as 'best_model.pkl'
CountVectorizer saved as 'count_vectorizer.pkl'

```
In [31]: import pickle

# Load model
with open("best_model.pkl", "rb") as f:
    loaded_model = pickle.load(f)

# Load vectorizer
with open("count_vectorizer.pkl", "rb") as f:
    loaded_cv = pickle.load(f)

# Predict on new data
new_data = ["your new text here"]
new_transformed = loaded_cv.transform(new_data)
prediction = loaded_model.predict(new_transformed)
print("Prediction:", prediction)
```

Prediction: [0]

```
In [32]: import pickle

# Load the saved model
with open(r"C:\Users\Lichh\best_model.pkl", "rb") as f:
    loaded_model = pickle.load(f)

print("Model loaded successfully!")
print("Model type:", type(loaded_model))
```

Model loaded successfully!
Model type: <class 'sklearn.linear_model._logistic.LogisticRegression'>

In []: